Практическая работа №6

Запросы на обновление REPLACE, UPDATE (DML)

Оператор языка SQL REPLACE позволяет атомарно добавлять в таблицу базы записи. Основное отличие операторов REPLACE и UPDATE в том, что при отработке первого ограничения удаляет запись (срабатывает ON DELETE CADCADE, если выставлено), что уступает в быстродействии IN DUPLICATE KEY UPDATE, который просто обновляет запись и не удаляет.

Перезапись значений столбца. Пример.

Записать в новый столбец данные. Реализовать задачу можно с помощью запроса на перезапись REPLACE INTO name_table VALUES (‘values’,’values’,’values’).

```
REPLACE INTO air VALUES ('0','azur air', '5');
```

Рисунок 6 – Перезапись с помощью запроса REPLACE

Также перезапись можно оформить с помощью запроса на обновление UPDATE name_table set name_column=new_value WHERE column1_name1=new_value

```
UPDATE `air` SET `airplane`=8 WHERE 'id_air'=2
```

Рисунок 7 – Перезапись с помощью запроса UPDATE

Задание:

С помощью запросов обоих видов добавить информацию в новые столбцы, задействовать оба способа.

Запрос на обновление текущей записи. Пример.

Обновить текущую запись в таблице возможно с помощью синтаксиса запроса **UPDATE name_table SET name_column = new_value**

[where…]

[order by…]

[limit …]

Запись с условием.

На примере запроса на обновление города и использование условия получается синтаксис, указанный на скриншоте. В данном случае город «Москва», который числился под первым номером, был заменен на город «Краснодар». WHERE используется для обозначения условия.

```
1 UPDATE `city` SET `id_city`='Краснодар' WHERE 'id_city'=1
```

Рисунок 8 – Обновление записи с помощью UPDATE и условия

Задание.

Создать 2 запроса на обновление записи с использованием условия. Например, снизить цену в 2 раза на ряд товаров.

Запись с упорядочиванием.

Обновить записи, добавив каждому порядковому номеру +1. Это может быть необходимо в тех случаях, когда необходимо вставить промежуточные значения. В примере в самом начале записей. ORDER BY name_column DESC(возрастание), ORDER BY name_column ASC (убывание).

```
UPDATE `passengers` SET `id_pass`=id_pass+1 order by id_pass desc
```

Рисунок 9 – Обновление с помощью упорядочивания по возрастанию

Задание.

Создать 2 запроса на обновление записей с использованием упорядочиванием записей по возрастанию и убыванию.

Запись с ограничением.

Необходимо обновить названия авиакомпаний, добавив на конце air, но сделать это необходимо только для 5 последних авиакомпаний. Для ограничения количества записей служит оператор LIMIT.

```
UPDATE `air` SET `name_air`=CONCAT(name_air, ' air') limit 5
```

Рисунок 10 – Обновление с помощью ограничения количества записей.

Задание.

Создать 2 запроса на обновление с ограничением количества записей.

Обновление с условием

В поле с комментарием разместить особые отметки о пассажире. Для этого необходимо вычислить является ли пассажир совершеннолетним или нет. Здесь поможет также функция вычисления разницы во времени, функция возвращающая текущую дату, а также условный оператор IF (условие, значение если истина, значение если ложь).

```
update passengers set comment=if(TIMESTAMPDIFF(year, date_of_birth, now())>=18, 'совершеннолетний','несовершеннолетний')
```

Рисунок 11 – Обновление с условием

Задание.

Создать 2 запроса на обновление с условием.

Условия также могут быть сложными. Необходимо рассчитать и записать скидку для определенных пассажиров в зависимости от их возраста. Известно, что дети до 2 лет летают бесплатно, дети от 2 до 18 со скидкой 40%, а взрослые не имеют скидки. Для того, чтобы описать такое условие можно воспользоваться оператором CASE.

```
update passengers set discount=CASE
when TIMESTAMPDIFF(year, date_of_birth, now())>=18 then 0
when TIMESTAMPDIFF(year, date_of_birth, now())<18 AND TIMESTAMPDIFF(year, date_of_birth, now())>=2 then 40
when TIMESTAMPDIFF(year, date_of_birth, now())<2  then 100
END
```

Рисунок 12 – Использование оператора CASE для обновления записей.

Задание

Создать 2 запроса на обновление с учетом условия CASE.

Функции, которые могут пригодиться при выполнении работы:

Функции даты-времени:

TIMESTAMPDIFF(interval, date1, date2) функция вычисляет разницу между датами date2 и date1 в единицах, указанных в параметре interval. Возможные значения параметра interval:

- FRAC_SECOND – микросекунды
- SECOND – секунды
- MINUTE – минуты
- HOUR – часы
- DAY – дни
- WEEK – недели
- MONTH – месяцы
- QUARTER - кварталы
- YEAR – годы

PERIOD_DIFF(period1, period2) функция вычисляет разницу в месяцах между двумя датами, представленными в формате YYYYMM

DATEDIFF(date1, date2) вычисляет разницу в днях между двумя датами.

TIMEDIFF(date1, date2) вычисляет разницу в часах, минутах и секундах между двумя датами.

PERIOD_ADD(period, n) функция добавляет n месяцев к значению даты period.

SUBDATE(date, INTERVAL value) функция идентична предыдущей, но производит операцию вычитания, а не сложения.

ADDDATE(date, INTERVAL value) Функция возвращает дату date, к которой прибавлено значение value.

CURDATE(), CURTIME() и NOW() Первая функция возвращает текущую дату, вторая - текущее время, а третья - текущую дату и время.

Строковые функции:

CONCAT: объединяет строки.

CONCAT_WS: также объединяет строки, но в качестве первого параметра принимает разделитель, который будет соединять строки

LEFT: вырезает с начала строки определенное количество символов. Первый параметр функции - строка, а второй - количество символов, которые надо вырезать сначала строки.

RIGHT: вырезает с конца строки определенное количество символов. Первый параметр функции - строка, а второй - количество символов, которые надо вырезать сначала строки

SUBSTRING(str, start [, length]): вырезает из строки str подстроку, начиная с позиции start. Третий необязательный параметр передает количество вырезаемых символов

REPLACE(search, find, replace): заменяет в строке find подстроку search на подстроку replace. Первый параметр функции - строка, второй - подстрока, которую надо заменить, а третий - подстрока, на которую надо заменить

LOWER: переводит строку в нижний регистр

UPPER: переводит строку в верхний регистр